

CELEBAL EXCELLENCE INTERNSHIP
PROGRAM

DATA ENGINEERING

WEEK - 6 ASSIGNMENT REPORT

SPARK ASSIGNMENT

SUBMITTED BY:-

ANUSHKA SHARMA

(JKLU)

➤ **OBJECTIVE**

The objective of this assignment is to understand the fundamentals of Apache Spark and gain hands-on experience with data loading, filtering, and transformation using Spark DataFrames.

It focuses on important Spark concepts such as lazy evaluation, in-memory processing, shuffle operations, and DataFrame immutability, which are essential for efficient big data processing. It also includes working with different file formats like CSV and Parquet, handling missing values, and applying basic data cleaning techniques.

In addition, the assignment involves building simple end-to-end data pipelines that follow a structured flow of reading, transforming, and writing data.

Overall, the assignment helps in understanding how Spark processes large datasets efficiently and how its architecture improves performance compared to traditional systems.

➤ **DATASET DESCRIPTION**

A sample dataset was generated in both CSV and Parquet formats, each containing 500 records. The dataset includes information such as user ID, transaction date, product category, price, region, and other sales-related attributes used for analysis and processing.

The dataset also contains duplicate records and null values to demonstrate data cleaning and transformation operations using Spark. Both CSV (row-based format) and Parquet (columnar format) were used to compare differences in storage structure and performance efficiency.

Spark Setup and Dataset Loading & Exploration

A Spark Session was created to initialize Apache Spark. This session acts as the entry point for performing all DataFrame operations such as reading files, transformations, and actions. The dataset was successfully loaded into Spark DataFrames from both CSV and Parquet files. Initial exploration was performed using methods like `show()`, `printSchema()`, and column inspection to understand the structure of the data before performing any further operations.

Setup and Data Loading

Import Spark libraries, create a Spark Session, and load the CSV and Parquet dataset for basic exploration.

```
[1] ✓ 0.9s
# Import required PySpark libraries
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *
```

```
# Create Spark Session
spark = SparkSession.builder \
    .appName("Week6 Spark Assignment") \
    .getOrCreate()
```

```
[3] ✓ 11.4s
# Load CSV file into DataFrame
df = spark.read.csv("data/source.csv", header=True, inferSchema=True)
```

```
[4] ✓ 1.0s
# Load Parquet file into DataFrame
parquet_df = spark.read.parquet("data/source.parquet")
```

```
# Display first 5 rows of CSV data
df.show(5)
```

[5] ✓ 0.6s

```
... +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|user_id|product_id|product_name| category| price|base_price| amount|quantity| supplier| status|region|priority|order_date|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| U3286| P792| Chair|Furniture| 173.8| 173.8| 869.0| 5| Vertex| Pending| North| High|2024-04-14|
| U4257| P818| Sofa|Furniture|1132.26| 1132.26|10190.34| 9|Global Ltd| Pending| East| High|2025-10-27|
| U4527| P204| T-Shirt| Clothing|3505.79| 3505.79|21034.74| 6| NextGen|Completed| East| Medium|2024-12-10|
| U7201| P665| Blocks| Toys|4045.28| 4045.28|32362.24| 8| Vertex| Pending| South| Low|2024-03-21|
| U2307| P987| Chair|Furniture| 276.83| 276.83| 1107.32| 4| NextGen| Pending| West| Low|2024-08-26|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows
```

```
# Display first 5 rows of Parquet data
parquet_df.show(5)
```

[6] ✓ 1.3s

```
... +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|user_id|product_id|product_name| category| price|base_price| amount|quantity| supplier| status|region|priority|order_date|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| U3286| P792| Chair|Furniture| 173.8| 173.8| 869.0| 5| Vertex| Pending| North| High|2024-04-14|
| U4257| P818| Sofa|Furniture|1132.26| 1132.26|10190.34| 9|Global Ltd| Pending| East| High|2025-10-27|
| U4527| P204| T-Shirt| Clothing|3505.79| 3505.79|21034.74| 6| NextGen|Completed| East| Medium|2024-12-10|
| U7201| P665| Blocks| Toys|4045.28| 4045.28|32362.24| 8| Vertex| Pending| South| Low|2024-03-21|
| U2307| P987| Chair|Furniture| 276.83| 276.83| 1107.32| 4| NextGen| Pending| West| Low|2024-08-26|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows
```

```
# Print schema of CSV DataFrame
df.printSchema()
```

[7] ✓ 0.0s

```
... root
  |-- user_id: string (nullable = true)
  |-- product_id: string (nullable = true)
  |-- product_name: string (nullable = true)
  |-- category: string (nullable = true)
  |-- price: double (nullable = true)
  |-- base_price: double (nullable = true)
  |-- amount: double (nullable = true)
  |-- quantity: integer (nullable = true)
  |-- supplier: string (nullable = true)
  |-- status: string (nullable = true)
  |-- region: string (nullable = true)
  |-- priority: string (nullable = true)
  |-- order_date: date (nullable = true)
```

```
# Count total number of rows in CSV DataFrame
df.count()
```

[8] ✓ 0.8s

... 500

```
# Show column names in DataFrame
df.columns
```

[9] ✓ 0.0s

... ['user_id',
 'product_id',
 'product_name',
 'category',
 'price',
 'base_price',
 'amount',
 'quantity',
 'supplier',
 'status',
 'region',
 'priority',
 'order_date']

```
# Get distinct values from 'category' column
df.select("category").distinct().show()
```

[10] ✓ 0.9s

... +-----+
 | category|
 +-----+
 | Sports|
 | Groceries|
 | Electronics|
 | Clothing|
 | Books|
 | Furniture|
 | Beauty|
 | Toys|
 +-----+

➤ ASSIGNMENT QUESTIONS

Q1: Explain the roles of the **Driver**, **Cluster Manager**, and **Executor** in a Spark application.

Answer:

The **Driver** is the main control program of a Spark application. It is where your Spark code starts running. It converts the code into a logical plan and then into a physical execution plan. It also divides the work into tasks and coordinates everything. It is like the "**brain**" of the Spark application.

The **Cluster Manager** is responsible for managing resources in the cluster. Its job is to allocate CPU, memory, and other resources to Spark applications. It does not process data itself; instead, it decides where and how resources should be allocated. Examples of cluster managers include YARN, Kubernetes, and Spark Standalone.

The **Executor** is where the actual work happens. Executors run on worker nodes and execute the tasks assigned by the driver. They also store intermediate data in memory or on disk during processing. Since multiple executors work in parallel, Spark can process large datasets much faster.

In short, the Driver plans and manages the work, the Cluster Manager provides the required resources, and the Executors perform the actual data processing.

Q2: How does Spark's **Lazy Evaluation** strategy improve performance when chain-processing large datasets?

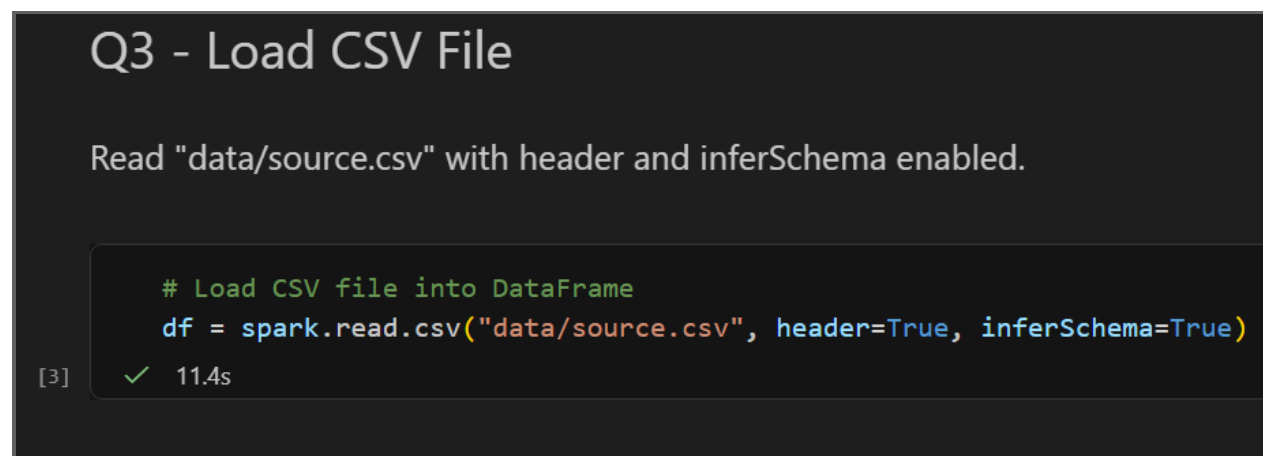
Answer:

Spark does not execute transformations immediately when we write them. Instead, it builds a logical execution plan called a **DAG (Directed Acyclic Graph)**. This behavior is called **lazy evaluation**.

Execution only happens when an action like `show()`, `count()`, or `write()` is called. This improves performance because Spark can optimize the entire workflow before execution. Instead of executing each transformation one by one, Spark combines multiple transformations into a single optimized execution plan. It also avoids unnecessary computations if some intermediate results are never used.

For large datasets, this results in less processing, less memory usage, and faster execution because Spark only does the minimum required work.

Q3: Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and `inferSchema` is enabled.

A screenshot of a Jupyter Notebook cell with a dark background. The title of the cell is "Q3 - Load CSV File". Below the title, the instruction "Read 'data/source.csv' with header and inferSchema enabled." is written. The code cell contains two lines of Python code: a comment "# Load CSV file into DataFrame" and the command "df = spark.read.csv('data/source.csv', header=True, inferSchema=True)". Below the code, the output shows "[3]" followed by a green checkmark and "11.4s", indicating successful execution.

```
Q3 - Load CSV File
```

Read "data/source.csv" with header and inferSchema enabled.

```
# Load CSV file into DataFrame
df = spark.read.csv("data/source.csv", header=True, inferSchema=True)
```

[3] ✓ 11.4s

Insight: The CSV file was loaded into a DataFrame with automatic schema detection. This makes sure all columns get correct data types, which helps in smooth processing later.

Q4: What is the difference between **CSV** and **Parquet** in terms of storage (row-based vs. columnar) and why does it matter for performance?

Answer:

CSV is a simple text-based format where data is stored **row by row**. Each line represents a complete record. Because of this structure, CSV files are easy to read and share, but they are not optimized for storage or performance.

Parquet, on the other hand, is a **columnar storage** format, which means it stores data **column by column** instead of row by row. This structure is much more efficient for big data processing.

When a query requires only specific columns, Spark reads only those columns instead of scanning the entire dataset. Parquet also compresses data more efficiently because similar data types are stored together.

As a result, Parquet uses less storage space, reads less data from disk, and executes queries much faster than CSV, especially when working with large datasets.

Q5: Given a DataFrame `df`, write a query to select the columns `product_id` and `price` where the category is 'Electronics'.

Q5 - Filter and Select Columns

Select `product_id` and `price` where category is 'Electronics'.

```
# Select only the required columns
electronics_df = df.filter(col("category") == "Electronics").select("product_id", "price")

# Display the result
electronics_df.show(5)
```

[11] ✓ 0.3s

```
... +-----+-----+
|product_id| price|
+-----+-----+
|      P165|4312.46|
|      P343|1183.26|
|      P919|3383.15|
|      P142|4181.25|
|      P984| 937.84|
+-----+-----+
only showing top 5 rows
```


Insight: Only required columns were selected after filtering the data for Electronics category. This reduces extra data and makes the output more focused and efficient.

Q6: Write the code to "revise" a DataFrame by renaming the column `old_name` to `new_name` and casting the price column from a String to a Double.

Q6 - DataFrame Update

Rename column `old_name("supplier")` to `new_name("vendor")` and cast price column to Double.

```
# Spark automatically converted 'price' to Double because we used inferSchema=True.
# For this question, we first convert it to String and then cast it back to Double
# to demonstrate the required operation.

df_q6 = df.withColumn("price", col("price").cast("string"))

# Rename the "supplier" column to "vendor"
# Convert price back to Double

df_q6 = df_q6.withColumnRenamed("supplier", "vendor") \
    .withColumn("price", col("price").cast("double"))

# Display the updated schema
df_q6.printSchema()

# Display a few records
df_q6.show(5)
```

[12] ✓ 0.3s

```

root
|-- user_id: string (nullable = true)
|-- product_id: string (nullable = true)
|-- product_name: string (nullable = true)
|-- category: string (nullable = true)
|-- price: double (nullable = true)
|-- base_price: double (nullable = true)
|-- amount: double (nullable = true)
|-- quantity: integer (nullable = true)
|-- vendor: string (nullable = true)
|-- status: string (nullable = true)
|-- region: string (nullable = true)
|-- priority: string (nullable = true)
|-- order_date: date (nullable = true)

+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|user_id|product_id|product_name| category| price|base_price| amount|quantity| vendor| status|region|priority|order_date|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| U3286| P792| Chair|Furniture| 173.8| 173.8| 869.0| 5| Vertex| Pending| North| High|2024-04-14|
| U4257| P818| Sofa|Furniture|1132.26| 1132.26|10190.34| 9|Global Ltd| Pending| East| High|2025-10-27|
| U4527| P204| T-Shirt| Clothing|3505.79| 3505.79|21034.74| 6| NextGen|Completed| East| Medium|2024-12-10|
| U7201| P665| Blocks| Toys|4045.28| 4045.28|32362.24| 8| Vertex| Pending| South| Low|2024-03-21|
| U2307| P987| Chair|Furniture| 276.83| 276.83| 1107.32| 4| NextGen| Pending| West| Low|2024-08-26|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows

```

Insight: The column name was changed and data type was converted to ensure consistency. This helps avoid errors while performing calculations or further operations.

Q7: How does Spark use the **Lineage Graph (DAG)** to provide fault tolerance if a worker node fails?

Answer:

Spark keeps track of all the transformations performed on a dataset in the form of a **Lineage Graph**, also known as a **DAG (Directed Acyclic Graph)**. This graph records how the data was created step by step from the original source.

If a worker node fails during execution, Spark does not lose the data permanently. Instead, it uses the lineage graph to identify the missing partitions and recomputes only those lost partitions by re-executing the required transformations.

This makes Spark highly fault tolerant because it does not need to restart the entire job. It only recreates the missing data, which saves both time and resources.

Q8: Write a query to filter a DataFrame `df_orders` for rows where the status is 'Completed' AND the amount is greater than 1000.

```
Q8 - Conditional Filtering

Filter df_orders where status is 'Completed' and amount > 1000.

# Filter rows where status is 'Completed' and amount is greater than 1000
completed_orders = df.filter((col("status") == "Completed") & (col("amount") > 1000))

# Display the filtered records
completed_orders.show(5)
```

✓ 0.3s Python

user_id	product_id	product_name	category	price	base_price	amount	quantity	supplier	status	region	priority	order_c
U4527	P204	T-Shirt	Clothing	3505.79	3505.79	21034.74	6	NextGen	Completed	East	Medium	2024-1
U4598	P432	Novel	Books	858.84	858.84	6011.88	7	Global Ltd	Completed	East	Medium	2025-1
U8019	P508	Blocks	Toys	741.17	741.17	6670.53	9	Global Ltd	Completed	West	High	2025-0
U2876	P649	Blocks	Toys	4856.84	4856.84	4856.84	1	Prime Supplies	Completed	East	Medium	2025-1
U9689	P713	Bread	Groceries	3824.43	3824.43	34419.87	9	NextGen	Completed	North	Medium	2024-0

only showing top 5 rows

Insight: The data was filtered using two conditions together. This helps in getting only important records that match the required business conditions.

Q9: Explain the concept of **Predicate Pushdown** in Parquet and how it affects the amount of data loaded into memory.

Answer:

Predicate Pushdown is an optimization technique where Spark applies the filter while reading the data, instead of loading all the data into memory first and then filtering it.

Since **Parquet** stores data in a columnar format and keeps information (metadata) about the data, Spark can quickly identify which parts of the file contain the required values. It reads only the relevant data and skips the rows or data blocks that do not match the filter condition.

For example, if we apply the filter **region = "North"**, Spark reads only the data related to the **North** region instead of reading the entire file first and then filtering it.

As a result, less data is read from disk, less memory is used, and the query runs much faster, especially when working with large datasets.

Q10: Write a code snippet to add a new column `final_price` which is the `base_price` multiplied by 1.18 (18% tax).

```
Q10 - Derived Column

Create final_price as base_price * 1.18 (18% tax).

# Create a new column by adding 18% tax to base_price
price_df = df.withColumn("final_price", col("base_price") * 1.18)

# Display the required columns
price_df.select("product_name", "base_price", "final_price").show(5)

✓ 0.2s
```

product_name	base_price	final_price
Chair	173.8	205.084
Sofa	1132.26	1336.0667999999998
T-Shirt	3505.79	4136.8322
Blocks	4045.28	4773.4304
Chair	276.83	326.65939999999995

only showing top 5 rows

Insight: A new column was created using a simple calculation on existing data. This is useful for adding business rules like tax or final price directly in the dataset.

Q11: What is the difference between **Transformations** and **Actions**? Provide two examples of each.

Answer:

In Spark, operations are divided into two types: Transformations and Actions.

Transformations are operations that create a new DataFrame or RDD from an existing one. They are lazy, which means Spark does not execute them immediately. Instead, it builds a plan (DAG) and waits until an action is called. Examples of transformations are `filter()` and `select()`.

Some transformations are **wide transformations**, like `groupBy()` or `join()`, where data is moved across partitions. This movement of data across partitions is called **Shuffle**, and it can make processing slower.

Actions are operations that trigger the execution of all transformations. They either return results or save the output. Examples of actions are `show()` and `count()`.

In short, transformations build the execution plan, while actions execute it and give the final result.

Q12: Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where `user_id` is null, and save the result as a CSV at "path/to/output".

Q12 - Parquet Processing

Read Parquet file, remove rows with null `user_id`, and save as CSV.

```
# The Parquet file was already loaded at the beginning of the notebook

# Filter rows where user_id is not null
filtered_df = parquet_df.filter(col("user_id").isNotNull())

# Save the filtered DataFrame as a CSV file
filtered_df.write \
    .mode("overwrite") \
    .option("header", True) \
    .csv("data/output-csv")

print("Filtered data saved successfully.")

# Total number of rows in the filtered DataFrame after removing null values in the user_id column
print(filtered_df.count())
```

✓ 1.2s

Filtered data saved successfully.
452

Insight: Unnecessary records were removed before saving the final output. This ensures the cleaned data is accurate and ready for further use.

Q13: In Spark Architecture, what is the difference between **Client Mode** and **Cluster Mode**?

Answer:

In **Client Mode**, the Spark **Driver** runs on the local machine from which the application is submitted. This mode is mainly used for development and debugging because the logs can be easily accessed on the local system.

In **Cluster Mode**, the Driver runs inside the cluster instead of the local machine. This mode is commonly used in production because the application continues running even if the local machine disconnects.

Q14: Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.

Q14 - OR Filtering

Filter rows where region is 'North' or priority is 'High'.

```
# Filter rows where region is 'North' OR priority is 'High'
region_priority_df = df.filter((col("region") == "North") | (col("priority") == "High"))

# Display the filtered records
region_priority_df.show(5)
```

✓ 0.5s

user_id	product_id	product_name	category	price	base_price	amount	quantity	supplier	status	region	priority	or
U3286	P792	Chair	Furniture	173.8	173.8	869.0	5	Vertex	Pending	North	High	20
U4257	P818	Sofa	Furniture	1132.26	1132.26	10190.34	9	Global Ltd	Pending	East	High	20
U6155	P771	Car	Toys	377.63	377.63	3776.3	10	NextGen	Cancelled	West	High	20
U8019	P508	Blocks	Toys	741.17	741.17	6670.53	9	Global Ltd	Completed	West	High	20
U3927	P208	Jacket	Clothing	2295.93	2295.93	11479.65	5	Vertex	Cancelled	South	High	20

only showing top 5 rows

Insight: Data was filtered using OR condition to include multiple possibilities. This makes it easier to capture all relevant records in one query.

Q15: When exploring a dataset, why is it safer to use `.show(5)` instead of `.collect()` on a multi-terabyte dataset?

Answer:

When working with very large datasets, using **`collect()`** is risky because it brings the entire dataset into the Driver's memory. For a multi-terabyte dataset, this can consume a huge amount of memory and may cause the application to slow down or even crash.

On the other hand, **`show(5)`** displays only the first five rows of the dataset. It does not load the entire dataset into the Driver's memory, making it much safer for exploring large datasets.

Therefore, `show()` is the preferred choice for viewing sample data, while `collect()` should only be used when the dataset is small enough to fit into the Driver's memory.

➤ CONCLUSION

In this assignment, Apache Spark DataFrames were used to perform data loading, cleaning, transformation, and analysis on structured datasets. Key operations such as duplicate removal, handling missing values, filtering, and aggregation were implemented successfully.

The assignment also helped in understanding important Spark concepts like lazy evaluation, shuffle operations, in-memory processing, and the difference between CSV and Parquet formats. Overall, it improved practical understanding of building efficient data pipelines using Spark.